code4rena

Reflector

Smart Contract Security Assessment

Audit dates: Oct 27 — Nov 11, 2025

Overview

About C4

Code4rena (C4) is a competitive audit platform where security researchers, referred to as Wardens, review, audit, and analyze codebases for security vulnerabilities in exchange for bounties provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the Reflector V3 smart contract system. The audit took place from October 27 to November 11, 2025.

Final report assembled by Code4rena.

Summary

The C4 analysis yielded an aggregated total of 6 unique vulnerabilities. Of these vulnerabilities, 1 received a risk rating in the category of HIGH severity and 5 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 54 QA reports compiling issues with a risk rating of LOW severity or informational.

All of the issues presented here are linked back to their original finding, which may include relevant context from the judge and Reflector team.

Scope

The code under review can be found within the [C4 Reflector V3 repository](), and is composed of 14 smart contracts written in the Rust programming language and includes 1,201 lines of Rust code.

The code in C4's Reflector repository was pulled from:

- Repository: [https://github.com/reflector-network/reflector-contract]()
- Commit hash: `ba7a401ee2f403855c844ab6c5072bc3925040a1`

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/informational.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges

- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on the C4 website, specifically our section on Severity Categorization.

High Risk Findings (1)

[H-01] `set_invocation_costs_config()` fails to authorize admin allowing anyone to set invocation costs

Submitted by YouCrossTheLineAlfie, also found by 0x_kmr_, 0xbrett8571, 0xgeeee, 0xkrodhan, 0xnija, 0xpetern, 0xshdax, 0xsolisec, 0xvd, AllTooWell, Almanax, ameng, Angry_Mustache_Man, arturtoros, aster, august1_, axelot, Bale, BioMatriX, cd_pandora, ChainSentry, CoMMaNDO, CowBoy, Dest1ny_rs, djshan_eden, dmdg321, edoscoba, escrow, eta, felconsec, foxb868, fullstop, Ganesh_197, HalalAudits, hecker_trieu_tien, holtzzx, ht111111, HUNTERRRRRRR, hyp3rion123, inh3l, jerry0422, Jesse, johnyfwesh, Josh4324, jsmaxi, JustUzair, K42, kimnoic, Kirkeelee, kjc, klau5, kwad, l3gb, luckygru, lufP, Mahmud, Manosh19, markoliver, marsspaceX, mbuba666, merlin, merlin_san, Mhayatt, mrdafidi, Mrunal2610, Mylifechangefast_eth, nathan47, NexusAudits, niffylord, NovaTheMachine, OhmOudits, oxwhite, piki, pv, rare_one, rhaloh_ke, rubencrxz, shaflow2, SiddiqX786, sl1, slavina, slvDev, swordfish, teoslaf, th3_hybrid, TheCarrot, touristS, trilobyteS, unique, Wojack, y4y, yixuan, zcai, zubyoz, and zzkiel

`lib.rs` #L404

Finding description and impact

The `set_invocation_costs_config()` function in `beam-contract/lib.rs` is designed to allow the admin to set invocation costs which is charged when prices are read by the consumer:

```
// Update costs configuration per each invocation category
// Requires admin authorization
//
// # Arguments
//
// * `config` - Invocation costs for different invocation categories
//
// # Panics
//
// Panics if not authorized or not initialized yet
pub fn set_invocation_costs_config(e: &Env, config: Vec<u64>) {
```

```
        set_costs_config(e, &config);
    }
```

However, this function fails to verify if the caller is admin; allowing anyone to set invocation costs.

This leads to unwarranted loss of funds to the price consumers as their tokens would get burnt as per invocation fee, which can be changed to a irrationally high number by a malicious actor.

Recommended mitigation steps

It is recommended to add a `auth::panic_if_not_admin(e);` check in order to mitigate this issue.

Proof of Concept

Add the following test case inside `beam-contract/src/test.rs`:

```
#[test]
fn anyone_can_set_invocation_config_test() {
    let (env, client, _) = init_contract_with_admin();

    env.mock_all_auths();

    let costs = Vec::from_array(&env, [10, 20, 30, 40, 50]);
    client.set_invocation_costs_config(&costs);

    let result = client.invocation_costs();
    assert_eq!(result, costs);
}
```

Medium Risk Findings (5)

[M-01] Systematic overcharge in `prices` and `x_prices`: Fee charged for requested records while return is capped at 20

Submitted by ht111111, also found by 0xdaxn, 0xgeeee, 0xnija, 0xpetern, 0xvd, AllTooWell, arturtoros, axelot, ayden, cd_pandora, CoMMaNDO, CowBoy, dmdg321, edoscoba, HalalAudits, Ibukun, inh3l, johnyfwesh, JustUzair, KKKKK, manaalwk, Manosh19, max10afternoon, merlin, mrdafidi, mrFreedom, NexusAudits, niffylord, OhmOudits, psyone, rare_one, shaflow2, shieldrey, sl1, soloking, teoslaf, wafflewizard, y4y, and zcai

- `lib.rs` #L208
- `cost.rs` #L88
- `prices.rs` #L215

Finding description

The root cause of the vulnerability is a mismatch between the fee calculation logic in the `beam-contract` and the data retrieval logic in the underlying `oracle` contract for the `prices` and `x_prices` functions.

1. **Fee Calculation**: The `beam-contract`'s `charge_invocation_fee` function calculates the service fee based directly on the `records` parameter provided by the user. The fee scales with the number of requested records.
2. **Data Retrieval**: When the `beam-contract` calls the `oracle` to fetch the historical price data, the `oracle`'s `load_prices` function silently caps the `records` parameter to a maximum of 20. Any request for more than 20 records will only return 20, without any error or notification to the caller.

This creates a situation where for example, a user can request 50 records and be charged a fee calculated for 50 records, but only receive 20 records in return. The discrepancy between the number of records paid for and the number of records received constitutes a systemic overcharging vulnerability.

Impact

The primary impact is a **direct and irrecoverable loss of user funds**. Users who call `prices` or `x_prices` with a `records` value greater than 20 are systematically overcharged. The excess fees paid are burned, meaning they cannot be recovered.

- **Scaling Financial Loss**: The magnitude of the overcharge increases linearly with the number of requested records. As demonstrated in the POC, requesting 100 records results in being overcharged by 333.3%, paying more than four times the fair price for the data received.
- **Hidden Bug**: The vulnerability is non-reverting and silent. Users receive a successful response with 20 data points and may not notice the discrepancy unless they carefully audit their token balance against the expected cost for the data they actually received.
- **Potential for Exploitation**: A malicious front-end could intentionally use high `records` values in its calls to the contract, causing users to burn excessive amounts of their tokens without their knowledge.

Recommended mitigation steps

To mitigate this vulnerability, the fee calculation should be synchronized with the actual number of records returned by the oracle. The most direct and least disruptive fix is to apply

the same cap in the `beam-contract` before calculating the fee.

In `beam-contract/src/lib.rs`, modify the `prices` and `x_prices` functions to cap the `records` parameter at 20 before passing it to `charge_invocation_fee`.

Example for the `prices` function:

```
:206:210:beam-contract/src/lib.rs
// ... existing code ...
pub fn prices(e: &Env, caller: Address, asset: Asset, records: u32) ->
Option<Vec<PriceData>> {
    caller.require_auth();

    // Mitigated fee charge
    let records_to_charge = records.min(20);
    charge_invocation_fee(e, &caller, InvocationComplexity::Price,
records_to_charge);

    PriceOracleContractBase::prices(e, asset, records)
}
// ... existing code ...
```

By changing `charge_invocation_fee(e, &caller, InvocationComplexity::Price, records);` to use `records.min(20)`, the fee charged will accurately reflect the maximum number of data points the user can receive, eliminating the overcharge issue. The same logic should be applied to the `x_prices` function.

Proof of Concept

[View detailed Proof of Concept]

[M-02] Expiration vector length mismatch causes panic in `extend_ttl()` when assets are added with zero initial expiration period

Submitted by piki, also found by 0xgeeee, 0xnija, 0xvd, Angry_Mustache_Man, Bala1796, CowBoy, cy97, HUNTERRRRRRR, jectaw, jsmaxi, KKKKK, klau5, Mahmud, Manosh19, newspacexyz, niffylord, nstatoshi, OhmOudits, oxwhite, Petrus, rare_one, shaflow2, sl1, and sudais_b

- `assets.rs` #L54-L80
- `assets.rs` #L109-L161
- `lib.rs` #L362-L364
- `assets.rs` #L93-L106

Finding description

The oracle contract maintains two parallel vectors: `asset_list` (all assets) and `expiration` (expiration timestamps). These vectors must always have the same length because `extend_ttl()` uses asset indices to access expiration records. However, a bug in `add_assets()` allows assets to be added without corresponding expiration records, breaking this invariant and causing `extend_ttl()` to panic.

The core problem: When `add_assets()` is called with `initial_expiration_period == 0`, the function adds the asset to `asset_list` but skips adding an expiration record to the `expiration` vector. This happens because of a conditional check that only creates expiration records when both the fee config is set AND the expiration timestamp is greater than zero.

Here's what happens step by step:

1. **Asset addition** (line 68): The asset is unconditionally added to `asset_list` via `asset_list.push_back(asset)`.
2. **Expiration record creation** (lines 70-72): The code checks:

```
if is_fee_config_set && expiration_timestamp > 0 {
    expiration.push_back(expiration_timestamp);
}
```

3. **The mismatch**: When `initial_expiration_period == 0`:
   - `get_expiration_timestamp()` returns `0` (line 17).
   - The condition `is_fee_config_set && expiration_timestamp > 0` evaluates to false.
   - Asset is added but NO expiration record is created.
   - Result: `asset_list.len() == N` but `expiration.len() == N-1`.

Why This breaks `extend_ttl()`: The `extend_ttl()` function assumes both vectors are in sync.

When it tries to update an expiration record:

1. It resolves the asset index from the asset list (line 121-125).
2. Loads the expiration vector (line 146).
3. Attempts to get the current expiration: `expiration.get(asset_index)` (line 148-150).
   - This returns `None` if the index is out of bounds, but the code handles this with `unwrap_or_else`.
4. **The crash happens in** (line 158): `expiration.set(asset_index, asset_expiration)`.
   - Soroban's `Vec::set()` panics with `IndexBounds` error when `asset_index >= expiration.len()`.
   - The error message confirms: `"object index out of bounds"`.

Real-world scenario: The Beam contract specifically calls `add_assets()` with `initial_expiration_period == 0` (see `beam-contract/src/lib.rs:363`). This means every time an admin adds assets through the Beam contract after setting a fee config, those assets are added without expiration records. When users try to extend the TTL for these assets by burning tokens, the transaction panics and fails.

Why `init_expiration_config()` doesn't fix it: The `init_expiration_config()` function seems like it should repair mismatches, but it has an early return check (line 95-96):

```
if expiration_records.len() > 0 {
    return; // expiration values for existing price feeds already
initialized
}
```

This means if ANY expiration records exist (even if incomplete), the function returns immediately without checking if the vectors are actually synchronized. So if you have 3 assets but only 2 expiration records, `init_expiration_config()` won't fix the mismatch.

Impact

1. **Functionality breakage**: Users cannot extend TTL for assets added via Beam contract's `add_assets()`. This breaks a core feature where sponsors burn tokens to keep price feeds alive.
2. **No work-around**: There's no way for users to fix this themselves. The admin would need to remove and re-add assets with a non-zero expiration period, but this isn't practical and may not be possible depending on contract state.
3. **Production risk**: This affects all assets added after fee config is set via Beam contract. In a production deployment, this could affect multiple assets and make them unusable for TTL extension.
4. **Silent failure**: The bug only manifests when someone tries to extend TTL. Assets can be added successfully, queries work fine, but the TTL extension feature is completely broken for those assets.

Attack Surface: While this isn't directly exploitable by attackers (it requires admin actions), it creates a situation where:

- Admin adds assets thinking everything is fine.
- Users try to sponsor price feeds by extending TTL.
- Transactions fail, potentially causing confusion and loss of funds (if gas/fees are charged).
- Price feeds may expire unexpectedly if TTL cannot be extended.

Recommended mitigation steps

The fix needs to ensure both vectors stay synchronized. Here are the recommended approaches:

Option 1: Always create expiration records when fee config is set. (Recommended)

Modify `add_assets()` to always create expiration records when fee config is set, even if `initial_expiration_period == 0`:

```
pub fn add_assets(e: &Env, assets: Vec<Asset>, initial_expiration_period:
u32) {
    let expiration_timestamp = get_expiration_timestamp(e,
initial_expiration_period);
    let mut asset_list = load_all_assets(e);
    let mut expiration = load_expiration_records(e);
    let is_fee_config_set = settings::get_fee_config(e) !=
FeeConfig::None;

    for asset in assets.iter() {
        if resolve_asset_index(e, &asset).is_some() {
            panic_with_error!(&e, Error::AssetAlreadyExists);
        }
        set_asset_index(e, &asset, asset_list.len());
        asset_list.push_back(asset);

        // FIX: Always create expiration record when fee config is set
        if is_fee_config_set {
            // Use expiration_timestamp if > 0, otherwise use current
time + initial_expiration_period
            let exp_time = if expiration_timestamp > 0 {
                expiration_timestamp
            } else {
                timestamps::ledger_timestamp(&e) +
timestamps::days_to_milliseconds(initial_expiration_period)
            };
            expiration.push_back(exp_time);
        }
    }
    // ... rest of function
}
```

Option 2: Add defensive check in `extend_ttl()`.

Extend the vector if needed before setting:

```
// In extend_ttl(), before line 158:
```

```
let mut expiration = load_expiration_records(e);
let all_assets = load_all_assets(e);

// Ensure expiration vector has enough capacity
while expiration.len() <= asset_index {
    expiration.push_back(0); // or appropriate default
}

expiration.set(asset_index, asset_expiration);
```

Option 3: Fix `init_expiration_config()` to repair mismatches.

Remove the early return and always ensure vectors are synchronized:

```
pub fn init_expiration_config(e: &Env, initial_expiration_period: u32) {
    let mut expiration_records = load_expiration_records(e);
    let assets = load_all_assets(e);
    let exp = get_expiration_timestamp(e, initial_expiration_period);

    // FIX: Always sync vectors, don't early return
    // Extend expiration records if needed
    while expiration_records.len() < assets.len() {
        expiration_records.push_back(exp);
    }

    set_expirations_records(e, &expiration_records);
}
```

Proof of Concept

Copy-paste PoC (drop-in test)

[View detailed Proof of Concept]

Expected test output: When running `poc_expiration_vector_mismatch_panic`, you should see:

```
thread 'tests::poc_expiration_vector_mismatch_panic' panicked at
.../host.rs:861:9:
HostError: Error(Object, IndexBounds)

Event log shows:
"object index out of bounds", 10
```

This confirms that `expiration.set()` panics when the index is out of bounds, proving the vector mismatch bug exists and causes real failures in production scenarios.

[M-03] `load_prices` function returns an incomplete list of prices

Submitted by newspacexyz, also found by 0x18a6, arturtoros, ayden, and Mylifechangefast_eth

`prices.rs` #L219-L227

Finding description and impact

When `get_price_fn` returns `None` (`retrieve_asset_price_data` or `load_cross_price` can return `None`), it skips to push the price to prices.

```
        if let Some(price) = get_price_fn(timestamp) {
                prices.push_back(price);
        }
```

But `records` is decreased and returns an incomplete list of prices. For example, when store prices skip one resolution, `retrieve_asset_price_data` returns `None`.

However, the user has already paid the fee for prices and gets an incomplete list of prices (fewer prices than expected, but paid for full prices vector). User has to pay more fees.

Also, `calculate_twap` returns `None` even though user has already paid fees.

Recommended mitigation steps

`load_prices` must not decrease records when `get_price_fn` returns `None`. Or, `BeamOracleContract::prices/x_prices` has to burn fee for `prices.len()`.

[M-04] `twap()` under-charges for multi-period queries due to hardcoded `periods=1`

Submitted by Sparrow, also found by 0x_kmr_, 0x1998, 0xgeeee, 0xnija, 0xpetern, 0xvd, Albort, boodieboodieboo, ChainSentry, CoMMaNDO, cryptoWhale, Daniel526, Dest1ny_rs, edoscoba, escrow, eta, Eurovickk, HalalAudits, hecker_trieu_tien, holtzzx, ht111111, HUNTERRRRRRR, iAfrika, inh3l, jectaw, jerry0422, Jesse, johnyfwesh, Josh4324, jsmaxi, JuggerNaut63, khaye26, Kirkeelee, KKKKK, kmkm, luckygru, markoliver, marsspaceX, mbuba666, Meks079, merlin, mrdafidi, Mylifechangefast_eth, NexusAudits, niffylord, OhmOudits, oxwhite, Petrus, piki, rare_one, shaflow2, sl1, slvDev, th3_hybrid, TheCarrot, touristS, trilobyteS, Wojack, y4y, yixuan, zcai, and zubyoz

`lib.rs` #L293-L298

Finding description

`twap()` and `x_twap()` pass a constant `1` to `charge_invocation_fee()` instead of the user-supplied `records` count. This causes massive under-charging for queries requesting multiple historical periods.

```
// In lib.rs
pub fn twap(e: &Env, caller: Address, asset: Asset, records: u32) ->
Option<i128> {
    caller.require_auth();
    charge_invocation_fee(e, &caller, InvocationComplexity::Twap, 1);  //
<-- Bug: always 1
    // ...
}
```

Impact

- Users requesting `TWAP` over `N` periods pay only the single-period fee.
- Predictable revenue leak; attackers can query with high `records` to minimize costs.

Each `TWAP`/`X-TWAP` call that requests `>1` period is under-billed by a factor of `N / 1`. Heavy integrators (bots, aggregators) can reduce their operational costs nearly to zero by always using large `records` values.

Proof of concept

```
// Caller wants TWAP over 15 rounds but pays for 1
let fee_before = BeamOracleContractClient::estimate_cost(
    &InvocationComplexity::Twap,
    &1,   // what contract *believes* is requested
);
let twap = BeamOracleContractClient::twap(&caller, &asset, &15);
// Internally charge_invocation_fee was called with periods = 1
```

Recommended mitigation steps

Pass the actual `records`:

```
charge_invocation_fee(e, &caller, InvocationComplexity::Twap, records);
```

[M-05] `x_last_price` uses global timestamp incorrectly

Submitted by ***HUNTERRRRRRR***

`price_oracle.rs` **#L218**

`x_last_price` looks up the *global* latest timestamp and tries to compute a cross-price at that exact tick:

```
pub fn x_last_price(e: &Env, base_asset: Asset, quote_asset: Asset) ->
Option<PriceData> {
    let timestamp = prices::obtain_last_record_timestamp(&e);   // global
last tick
    if timestamp == 0 {
        return None;
    }
    let decimals = settings::get_decimals(e);
    let asset_pair_indexes = assets::resolve_asset_pair_indexes(e,
base_asset, quote_asset)?;
    prices::load_cross_price(&e, asset_pair_indexes, timestamp, decimals)
}
```

If the most recent snapshot updated only one of the two assets (partial update), one side of the pair may have no record at that timestamp, so `load_cross_price` returns `None` even though a valid cross-price *does* exist at the previous timestamp; where both assets were present.

In effect, `x_last_price` can intermittently return `None` (or fail upstream logic) right after a partial update. This is a critical logic flaw for consumers that rely on a "latest" cross-price.

- **Availability/DoS risk:** A price publisher that posts a partial snapshot (only base *or* quote) causes `x_last_price` to report "no price," potentially halting trading or causing fallbacks.
- **Inconsistency:** Other functions (`x_prices`, `x_twap`) correctly scan backward over history via `prices::load_prices`, so the "latest" behavior differs depending on which API is called.

Low Risk and Informational Issues

For this audit, 54 QA reports were submitted by wardens compiling low risk and informational issues. The QA report highlighted below by **Angry_Mustache_Man** received the top score from the judge. 17 Low-severity findings were also submitted individually, and can be viewed here.

The following wardens also submitted QA reports: 0x_DyDx, 0xbrett8571, 0xenzo_eth, 0xki, 0xnija, 0xshdax, Abdulyb, amirhossineedalat, Anas4audits, aster, Astroboy, bam0x7, Bluedragon101, ChainSentry, cosin3, dee24, Ephraim, eta, Eurovickk, foxb868, francoHacker, gigantic, hyp3rion123, johnyfwesh, jsmaxi, JustUzair, K42, kestyvickky, khaye26, KineticsOfWeb3, LeopoldFlint, luckygru, mbuba666, Meks079, NexusAudits, niffylord, Petrus, phR35h, pv, Race, rare_one, Rorschach, ryzen_xp, sabby, shieldrey, totdking, trilobyteS, unique, valarislife, Xmannuel, y4y, zcai, and zubyoz.

[01] README documentation mismatch: Asset limit discrepancy

The README documentation states that each oracle contract can support up to 256 assets, but the actual implementation uses a limit of 1000 assets. However, the update record mask implementation is fundamentally limited to 256 assets, making the 1000 asset limit in the code incorrect and potentially causing runtime errors. This creates a three-way discrepancy: documentation says 256, code constant says 1000, but the actual technical implementation only supports 256.

Location

- `README.md` #L154: "Each oracle contract can support up to 256 assets and retain up to 256 historical update records"
- `oracle/src/assets.rs` #L5: `const ASSET_LIMIT: u32 = 1000; //current limit.`
- `oracle/src/assets.rs` (usage) #L74-L76: The check uses `ASSET_LIMIT` which is 1000.
- `oracle/src/mapping.rs` L#65-L73: The `resolve_period_update_mask_position()` function uses a 256-bit (32-byte) update record mask, which can only track up to 256 assets. The function calculates byte position as `asset_index / 8`, meaning:
  - Assets 0-255 → Bytes 0-31 (fits in 32-byte mask)
  - Assets 256+ → Byte 32+ (out of bounds for 32-byte mask)

Technical Limitation

The update record mask used in price update records is limited to 256 bits (32 bytes), as indicated by the comment "256-bit update record mask" in `mapping.rs`. When an asset with index 256 or higher is added, the `resolve_period_update_mask_position()` function will calculate a byte position beyond the 32-byte mask boundary, potentially causing:

- Out-of-bounds access when checking if an asset was updated in a period.
- Incorrect tracking of which assets have price updates.
- Potential panics or undefined behavior when processing price updates for assets beyond index 255.

Examples

- Asset at index 256: `byte = 256 / 8 = 32`, but the mask is only 32 bytes (indices 0-31).
- Asset at index 500: `byte = 500 / 8 = 62`, which is far beyond the mask size.
- Asset at index 999: `byte = 999 / 8 = 124`, which is completely out of bounds.

Recommended mitigation steps

The code constant `ASSET_LIMIT` should be reduced to 256 to match both the documentation and the technical limitation of the update record mask. The current 1000 limit is misleading and can lead to runtime errors when assets beyond index 255 are added. Alternatively, if 1000 assets are truly needed, the update record mask implementation would need to be redesigned to support 1000 assets (requiring 125 bytes = 1000 bits).

[02] Incorrect minimum ledger threshold logic in TTL extension

The code comment states that "16 ledgers is the minimum extension period" for TTL extension, but the implementation uses a strict greater-than comparison (`ledgers_to_live > 16`) instead of greater-than-or-equal (`ledgers_to_live >= 16`). This means when `ledgers_to_live` equals exactly 16, the TTL extension is not performed, contradicting the documented minimum requirement. This inconsistency can lead to unexpected behavior where the minimum threshold is not actually enforced as documented.

Location

- `oracle/src/prices.rs` (`store_prices`) #L189-L192:

```
if ledgers_to_live > 16 {
  //16 ledgers is the minimum extension period
  temps_storage.extend_ttl(&timestamp, ledgers_to_live, ledgers_to_live)
}
```

- `oracle/src/prices.rs` (`store_price_v1`) #L307-L310: Same issue in the `store_price_v1()` function.

Impact

When `ledgers_to_live` is calculated to be exactly 16, the TTL extension is skipped, which may cause price records to expire earlier than expected. This could lead to data loss or unavailability of price records that should have been extended according to the documented minimum.

Recommended mitigation steps

Change the comparison from `ledgers_to_live > 16` to `ledgers_to_live >= 16` to properly enforce the documented minimum extension period of 16 ledgers. Alternatively, if the minimum should be exclusive, update the comment to clarify that the minimum is actually 17 ledgers.

[03] TWAP strict length check causes complete failure on partial data

The `calculate_twap()` function requires that the number of returned price records exactly matches the requested number. If `load_prices()` returns fewer records than requested (due to the 20-record limit, missing historical data, or early termination), the function returns `None` completely, even if sufficient data exists for a valid TWAP calculation.

Location

`oracle/src/prices.rs` #L243-L247:

```
let prices = load_prices(&e, get_price_fn, records)?;

if prices.len() != records {
  return None;
}
```

Impact

This strict check causes TWAP calculations to fail completely when:

- User requests more than 20 records (limited by `load_prices()`).
- Some historical price data is missing (sparse updates).
- Early termination occurs in `load_prices()` due to timestamp boundaries.

Even if 19 out of 20 requested records are available, the function returns `None` instead of calculating TWAP with available data. This creates a poor user experience where valid TWAP calculations are rejected due to minor data gaps, especially when combined with the upfront fee charging mechanism (users pay for the full request but get nothing if even one record is missing).

Recommended mitigation steps

Consider relaxing the strict check to allow TWAP calculation with available data, perhaps requiring a minimum threshold (e.g., at least 50% of requested records) rather than requiring exact match. Alternatively, document this strict requirement clearly so users understand that partial data will result in complete failure.

[04] Contract upgrade mechanism lacks timelock or delay

The `update_contract()` function allows the admin to upgrade the contract code immediately without any timelock, delay period, or community notification mechanism. This creates a risk where a compromised admin (or multisig majority) could deploy malicious code that takes effect immediately.

Location

`oracle/src/price_oracle.rs` **#L465-468**:

```
pub fn update_contract(e: &Env, wasm_hash: BytesN<32>) {
  auth::panic_if_not_admin(e);
  e.deployer().update_current_contract_wasm(wasm_hash);
}
```

Impact

A compromised admin (or multisig majority) could:

- Deploy malicious contract code that takes effect immediately.
- Bypass all security checks and authorization mechanisms.
- Manipulate prices, or cause other critical issues.
- Leave users with no time to react.

Recommended mitigation steps

Implement a timelock mechanism:

1. **Scheduled upgrades**: Require upgrades to be scheduled with a minimum delay (e.g., 7-14 days).
2. **Two-step process**: First propose the upgrade, then execute after the delay.
3. **Community notification**: Emit events when upgrades are proposed.
4. **Emergency upgrades**: Allow immediate upgrades only through a higher threshold (e.g., 80%+ multisig) for true emergencies.

[05] Admin controls all critical configuration parameters

The admin has unrestricted control over all critical configuration parameters including fee structures, asset lists, cache settings, and invocation costs. While protected by multisig, there are no limits, timelocks, or additional safeguards on these changes, creating centralization risks.

Location

`oracle/src/price_oracle.rs` multiple admin functions:

- `set_fee_config()` #L414-L418: Controls fee token and amounts.
- `add_assets()` #L383-386: Controls which assets are supported.
- `set_cache_size()` L367-L370: Controls caching behavior.
- `set_history_retention_period()` #L398-L401: Controls data retention.
- `set_invocation_costs_config()` (beam-contract): Controls invocation fees.

Impact

An admin (or compromised multisig majority) can:

- Change fee structures arbitrarily, potentially making the oracle unusable.
- Add or remove assets without community input.
- Manipulate caching to affect performance.
- Change data retention periods, potentially causing data loss.
- Set invocation costs to extreme values, blocking or enabling free access.

These changes can be made immediately without:

- Community notification or voting.
- Timelock delays for review.
- Limits on change magnitude.
- External validation or approval.

Recommended mitigation steps

Consider implementing:

1. **Change limits**: Restrict the magnitude of changes (e.g., fees can only change by `±20%` per update).
2. **Timelock delays**: Require delays for critical configuration changes.
3. **Gradual changes**: Implement gradual change mechanisms for sensitive parameters.
4. **Community governance**: Require community voting for major changes.
5. **Parameter bounds**: Enforce minimum/maximum bounds on all configurable parameters.

Comment from the Reflector team: the admin role is a multisig account, so several of these issues are design choices based on the precondition that for any such privileged action, the majority of 7 cluster organizations must provide their explicit permission.

Disclosures

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and disclosed to sponsoring

developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.